

.NET Applications

The Agentic AI Platform provides official NuGet packages for integrating the Trimble Agentic Chat UI into .NET desktop and mobile applications. All packages are built on top of `Trimble.AgenticChat.Core`, which handles the JSON message protocol between the host application and the Chat UI.

Available Packages

Package	Description
<code>Trimble.AgenticChat.WebView2</code>	For WPF / WinForms (.NET 8 or higher, .NET Framework 4.8.1)
<code>Trimble.AgenticChat.MAUI</code>	For .NET MAUI (Windows, Android, iOS, macOS)
<code>Trimble.AgenticChat.Core</code>	Contains core logic. Consumed by each framework-specific package

NuGet Feed Setup

Add the Trimble Agentic NuGet feed to your `nuget.config`:

```
<configuration>
  <packageSources>
    <add key="trimble-agentic-external-nuget-local"
      value="https://artifactory.trimble.tools/artifactory/api/nuget/v3/trimble-agentic-external-nuget-local" />
  </packageSources>
</configuration>
```

Core Concepts

Each framework specific wrapper requires the consuming application to implement the following set of interfaces:

IAgenticChatProvider — required

The single interface your application **must** implement. The wrapper calls these methods in response to messages from the Chat UI:

Method	Description
<code>ProvideChatUiConfig()</code>	Returns the initial <code>ChatUiConfiguration</code> (environment, agent, theme, locale, ...).
<code>ProvideChatUiToken()</code>	Returns a valid Trimble Identity bearer token for the current user.
<code>ProvideRunConfig(agentId)</code>	Called before each agent run. Return optional local tool definitions and run context.
<code>OnUnauthorized()</code>	Called when the UI reports an expired session. Return a refreshed token, or <code>null</code> to force sign-out.

ChatUiConfiguration

```
new ChatUiConfiguration
{
    Environment      = Environment.Stage, // Stage | Prod
    AgentId          = "your-agent-uuid",
    ThreadId         = null,              // optional: resume a
specific conversation
    OnBeforeRunTimeout = null,            // optional: ms timeout
for the onBeforeRun hook
    UiConfig = new UiConfig
    {
        Theme      = Theme.Light,        // Light | Dark
        Variant     = ChatUiVariants.Full, // Full | Minimal |
Narrow
        ContentVariant = ContentVariants.Chat, // Chat | AgentCards
    }
}
```

```
ChatInput      = new ChatInput
{
    Buttons      = new List<ChatInputButton>(),
    HideModelSelection = false
},
Localization = new Localization
{
    Locale = "en" // BCP-47 locale tag, e.g. "en", "de", "fr"
}
```

Optional handlers

Interface	Purpose
IAgenticChatEventHandler	Receives UI navigation events (OnAgentSelect, OnNewChat, OnClose, ...).
IAgenticChatExceptionHandler	Receives exceptions that occur during message handling.
IRawAgenticChatMessageHandler	Receives every raw JSON message for debugging. ⚠ Messages may contain sensitive info such as access tokens — do not persist them.

WebView2 (WPF / WinForms)

Trimble.AgentiChat.WebView2 wraps Trimble.AgentiChat.Core for applications that embed a Microsoft.Web.WebView2 control. It navigates the control directly to the Chat UI URL and wires up two-way WebMessage communication automatically.

Installation

```
dotnet add package Trimble.AgentiChat.WebView2
```

WPF Example

```
<!-- MainWindow.xaml -->
< Window xmlns: wv2 = "clr-
namespace:Microsoft.Web.WebView2.Wpf;assembly=Microsoft.Web.WebView2.Wpf" >
  < Grid >
    < wv2: WebView2 x: Name = " AgenticWebView" >
  />
</ Grid >
</ Window >
```

```
using Trimble.AgenticChat.Core;
using Trimble.AgenticChat.Core.AgenticModels;
using Trimble.AgenticChat.Core.Interfaces;
using Trimble.AgenticChat.Core.Models;
using Trimble.AgenticChat.WebView2;

public partial class MainWindow : Window
{
    private AgenticChatWebView2 _agenticChat;

    public MainWindow()
    {
        InitializeComponent();
        Loaded += OnLoadedAsync;
        AgenticWebView.EnsureCoreWebView2Async();
    }

    private async void OnLoadedAsync(object sender, RoutedEventArgs e)
    {
        await AgenticWebView.EnsureCoreWebView2Async();

        _agenticChat = new AgenticChatWebView2(
            provider:      new MyAgenticChatProvider(),
            eventHandler:  new MyEventHandler(),
            exceptionHandler: new MyExceptionHandler()
        );

        await _agenticChat.ListenToChatUi(AgenticWebView.CoreWebView2);
    }
}
```

WinForms Example

```
using Microsoft.Web.WebView2.WinForms;
using Trimble.AgenticChat.WebView2;

public class MainForm : Form
{
    private readonly WebView2 _webView = new WebView2 { Dock =
DockStyle.Fill };
    private AgenticChatWebView2 _agenticChat;

    public MainForm()
    {
        Controls.Add(_webView);
        Load += OnLoadAsync;
    }

    private async void OnLoadAsync(object sender, EventArgs e)
    {
        await _webView.EnsureCoreWebView2Async();

        _agenticChat = new AgenticChatWebView2(new
MyAgenticChatProvider());

        await _agenticChat.ListenToChatUi(_webView.CoreWebView2);
    }
}
```

Updating configuration at runtime

Call `UpdateConfig` to push a new `ChatUiConfiguration` to the UI without reloading — useful when the user changes the theme or agent:

```
_agenticChat.UpdateConfig(new ChatUiConfiguration
{
    Environment = Environment.Prod,
    AgentId      = "another-agent-uuid",
    UiConfig     = new UiConfig { Theme = Theme.Dark }
});
```

Cleanup

`AgenticChatWebView2` implements `IDisposable`. Dispose it to detach all event handlers:

```
_agenticChat?.Dispose();
```

MAUI

`Trimble.AgenticChat.MAUI` wraps `Trimble.AgenticChat.Core` for .NET MAUI applications. Because `HybridWebView` does not support loading remote URLs directly, the package includes a local HTML shell (`agentic-chat-shell.html`) that is automatically added to your project as a `MauiAsset`. The shell loads the remote Chat UI in an `iframe` and bridges `HybridWebView`'s `RawMessage` APIs with the `iframe`'s `postMessage` API.

```
C# host code ↔ HybridWebView (agentic-chat-shell.html) ↔ iframe  
(remote Chat UI)
```

Installation

```
dotnet add package Trimble.AgenticChat.MAUI
```

No manual `<MauiAsset>` entries are needed — the package's MSBuild targets handle this automatically.

XAML Setup

```
<!-- MainPage.xaml -->  
< ContentPage xmlns:maui="clr-namespace:Microsoft.Maui.Controls;assembly=Microsoft.Maui.Controls" >  
  < HybridWebView  
    x:Name="AgenticHybridWebView"  
    DefaultFile="agentic-chat-shell.html" />  
</ ContentPage >
```

Code-behind

```
using Trimble.AgenticChat.Core.Interfaces;  
using Trimble.AgenticChat.MAUI;
```

```

public partial class MainPage : ContentPage
{
    private AgenticChatMAUI _agenticChat;

    public MainPage()
    {
        InitializeComponent();

        protected override async void OnAppearing()
        {
            base.OnAppearing();

            if (_agenticChat != null) return;

            _agenticChat = new AgenticChatMAUI(
                provider:      new MyAgenticChatProvider(),
                eventHandler:   new MyEventHandler(),
                exceptionHandler: new MyExceptionHandler()
            );

            // On Windows, the handler must be attached before calling
            ListenToChatUi.
            // Wrap in a Dispatcher.DispatchAsync if needed.
            await _agenticChat.ListenToChatUi(AgenticHybridWebView);
        }
    }
}

```

i On Windows, `ListenToChatUi` requires the `HybridWebView` handler to be fully attached to the visual tree. If you call it during page construction, wrap it in `Dispatcher.DispatchAsync` or call it from a `Loaded` event handler.

Updating configuration at runtime

```

_agenticChat.UpdateConfig(new ChatUiConfiguration
{
    Environment = Environment.Prod,
    AgentId      = "another-agent-uuid",
    UiConfig     = new UiConfig { Theme = Theme.Dark }
});

```

Cleanup

`AgenticChatMAUI` implements `IDisposable`:

```
_agenticChat?.Dispose();  
_agenticChat = null;
```

Implement Your Own Wrapper Using `Trimble.AgenticChat.Core`

If neither `WebView2` nor `MAUI` fits your use-case (e.g. CefSharp, Avalonia WebView, or a custom IPC channel), you can drive the protocol directly using

`Trimble.AgenticChat.Core`.

Installation

```
dotnet add package Trimble.AgenticChat.Core
```

How it works

`AgenticChat` is the central class. You feed it raw JSON strings received from the web UI, and it returns a typed `BaseResponse` that you serialize and send back:

```
Your browser control → messageJson →  
AgenticChat.HandleReceivedMessageAsync()  
← responseJson ← Serialize(BaseResponse)
```

Step 1 — Implement `IAgenticChatProvider`

```
using Trimble.AgenticChat.Core;  
using Trimble.AgenticChat.Core.AgenticModels;  
using Trimble.AgenticChat.Core.Interfaces;  
using Trimble.AgenticChat.Core.Models;  
  
public class MyProvider : IAgenticChatProvider  
{  
    public Task<ChatUiConfiguration> ProvideChatUiConfig() =>
```



```

        Task.FromResult(new ChatUiConfiguration
        {
            Environment = Environment.Stage,
            AgentId      = "your-agent-uuid",
            UiConfig     = new UiConfig
            {
                Theme     = Theme.Light,
                Variant    = ChatUiVariants.Full
            }
        });

public Task<string> ProvideChatUiToken() =>
    // Return the Trimble Identity access token for the current
user
    Task.FromResult(GetCurrentAccessToken());

public Task<OnBeforeRunConfig> ProvideRunConfig(string agentId) =>
    Task.FromResult(new OnBeforeRunConfig());

public Task<string> OnUnauthorized() =>
    // Return a refreshed token, or null to force sign-out in the
UI
    Task.FromResult<string>(null);

private string GetCurrentAccessToken() => /* your token logic */
throw new NotImplementedException();
}

```

Step 2 — Implement `IAgenticChatEventHandler` (optional)

```

using Trimble.AgenticChat.Core.AgenticModels;
using Trimble.AgenticChat.Core.Interfaces;

public class MyEventHandler : IAgenticChatEventHandler
{
    public void OnEvent(ChatUiEvent evt)
    {
        Console.WriteLine($"UI event: {evt.Type}  payload:
{evt.Payload}");
        // evt.Type is one of: OnAgentSelect, OnThreadSelect,
OnNewChat,
        // OnExploreAgents, OnCreateAgent, OnMyTrimbleClick,
        // OnChatInputButtonClick, OnSignIn, OnClose
    }
}

```

```
}  
  
}
```

Step 3 — Wire up **AgenticChat** to your browser control

```
using Newtonsoft.Json;  
using Trimble.AgenticChat.Core;  
  
public class MyCustomWrapper : IDisposable  
{  
    private readonly AgenticChat _agenticChat;  
    private readonly IAgenticChatExceptionHandler _exceptionHandler;  
    private MyBrowserControl _browser;  
  
    public MyCustomWrapper(IAgenticChatProvider provider,  
                           IAgenticChatEventHandler eventHandler =  
null,  
                           IAgenticChatExceptionHandler  
exceptionHandler = null)  
    {  
        _agenticChat = new AgenticChat(provider, eventHandler);  
        _exceptionHandler = exceptionHandler;  
    }  
  
    public async Task ListenToChatUiAsync(MyBrowserControl browser)  
    {  
        _browser = browser;  
  
        // Navigate to the Chat UI URL for the configured environment  
        ChatUiConfiguration config = await  
_agenticChat.Provider.ProvideChatUiConfig();  
        Constants.ChatUiUrls.TryGetValue(config.Environment, out string  
url);  
  
        _browser.Navigate(url);  
  
        // Subscribe to incoming messages from the web UI  
        _browser.MessageReceived += OnMessageReceivedAsync;  
    }  
  
    private async void OnMessageReceivedAsync(object sender, string  
messageJson)  
    {  
        try  
        {  

```

```

        BaseResponse response = await
_agenticChat.HandleReceivedMessageAsync(messageJson);

        if (response != null)
        {
            // Serialize and post the response back to the web UI
            string responseJson =
JsonConvert.SerializeObject(response);
            _browser.PostMessage(responseJson);
        }
    }
    catch (Exception ex)
    {
        _exceptionHandler?.OnException(ex);

        // Attempt to send a typed error response back to the UI
        BaseResponse errorResponse =
_agenticChat.TryParseExceptionResponse(messageJson, ex);
        if (errorResponse != null)
        {
            _browser.PostMessage(JsonConvert.SerializeObject(errorResponse));
        }
    }
}

/// <summary>
/// Proactively push a configuration update to the UI at any time.
/// </summary>
public void UpdateConfig(ChatUiConfiguration config)
{
    BaseResponse response =
_agenticChat.GetConfigUpdateResponse(config);
    _browser.PostMessage(JsonConvert.SerializeObject(response));
}

public void Dispose() => _browser.MessageReceived -=
OnMessageReceivedAsync;
}

```

Local tools and run context

`ProvideRunConfig` is the place to register local tools (functions the AI can call in the host application) and inject per-run context:

```

public Task<OnBeforeRunConfig> ProvideRunConfig(string agentId)
{
    var tools = new LocalTools
    {
        RunTime = new Dictionary<string, RuntimeToolConfig>
        {
            ["get_current_selection"] = new RuntimeToolConfig
            {
                Definition = new Tool
                {
                    Name = "get_current_selection",
                    Description = "Returns the currently selected item
in the host application.",
                    InputSchema = new { type = "object", properties =
new { } }
                },
                Callback = async (args) =>
                {
                    // args is a Dictionary<string, JToken>
                    return JsonConvert.SerializeObject(new { id =
GetSelectedItemId() });
                },
                TimeoutInMs = 5000
            }
        }
    };

    var context = new OnBeforeRunConfig
    {
        Tools = tools,
        RunContext = new RunContext
        {
            Context = new List<ContextObject>
            {
                new ContextObject { Description = "Current user", Value
= GetCurrentUser() }
            }
        }
    };

    return Task.FromResult(context);
}

```